SAMPLE TIME AND TIMING ON THE ATMEGA328P

1.0 Learning Outcomes.....1
2.0 Hardware Set-Up.....1
3.0 Practical Investigation via Simulink.....1
4.0 Concluding Remarks.....4
Recommended Citation.....5

1.0 Learning Outcomes

Upon completing this practical exercise, students should be able to demonstrate the Learning Outcomes presented in Table 1. At the end of the exercise, revisit each Learning Outcome and confirm that it has been achieved.

Table 1: *Learning Outcomes for the Practical Exercise*

ID	Learning Outcome	Achieved
1	Build and deploy a Simulink model that reads a push-button and sets the brightness of an LED with PWM on the Arduino Uno.	☐
2	Explain how the sample time of a model decides which input events are detected, and measure the effect of the sample time on missed button presses.	☐
3	Detect task overruns on the ATmega328P, and explain why a controller needs a sample time that the processor can always meet.	☐

2.0 Hardware Set-Up

This practical exercise builds on the hardware set-up described in the practical exercises, LED (PWM) and Tactile Button Switch, which detail how an LED is driven from a PWM output and how a push-button is read as a digital input. The set-up is not repeated here.

The LED, with its series resistor, should be connected to a PWM-capable pin of the Arduino Uno. On the Uno, pins 5 and 6 produce PWM at about 980 Hz, and pins 3, 9, 10 and 11 at about 490 Hz. Note which pin you use, as the PWM frequency is needed later in the exercise.

3.0 Practical Investigation via Simulink

MATLAB and Simulink should be used to complete this practical exercise. A simple LED dimmer is built in which each press of the button increases the brightness of the LED. Changing the sample time of the model then shows how the ATmega328P samples its inputs, and how much computation it can complete in each sample period. The exercise supports the ACE-CORE learning guide, Introduction to the ATmega328P Microcontroller (Arduino Uno).

Background: Sampling an Input

A Simulink model deployed to the Arduino Uno runs once every sample period T_s. In each period, the inputs are read once, the model is calculated, and the outputs are written. Anything that happens to an input between two samples is not seen. A button press of duration T_p is therefore certain to be detected only if it lasts at least one sample period. A shorter press is detected only if a sample happens to fall during it:

$$P(\text{detected}) = \min(1,\ \frac{T_p}{T_s})$$

The theoretical chance of detecting a press is plotted in Figure 1 for three sample times. A quick press typically lasts 50 to 150 ms, so it is always detected at T_s = 0.01 s but is often missed at T_s = 0.5 s.

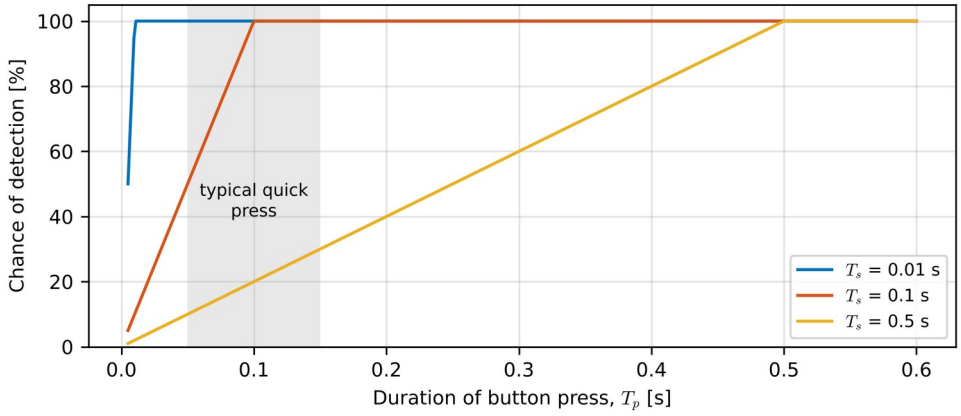


Figure 1: *Theoretical Chance of Detecting a Button Press against its Duration, for Three Sample Times*

Build the LED Dimmer

In the Simulink model, the button is read with a Digital Input block, and a Detect Rise Positive block turns each press into a single pulse. A MATLAB Function block counts the presses and sets the PWM value, which drives the LED through a PWM block, see Figure 2.

[Screenshot to insert] *Digital Input (button) → Detect Rise Positive → MATLAB Function (press counter) → PWM output (LED pin).*

Figure 2: *Simulink Model of the LED Dimmer*

MATLAB Function Block: counting button presses and setting the LED brightness

```
function duty = fcn(press)
% press: 1 for one sample at each new button press, otherwise 0
persistent n
if isempty(n)
    n = 0;                      % number of presses since the start
end
if press
    n = mod(n + 1, 11);         % 0 to 10 presses, then wrap back to 0
end
duty = round(n*25.5);           % PWM value: 0 (off) to 255 (full brightness)
```

1. Build the model in Figure 2 and set the sample time of the Digital Input block, and the fixed-step size of the model, to T_s = 0.01 s.
2. Deploy the model to the Arduino Uno. Press the button ten times and confirm that the brightness increases in ten steps.
3. Press the button ten times as quickly as you can, and count how many presses change the brightness. Record the number of missed presses in Table 2.
4. Repeat with T_s = 0.1 s and T_s = 0.5 s.

Table 2: *Missed Button Presses for Different Sample Times*

Sample time, T_s (s)	Presses detected (out of 10)	Presses missed	Detected with slow presses?
0.01			
0.1			
0.5			

Detect Task Overruns

The ATmega328P runs at 16 MHz and has no hardware support for floating-point arithmetic, so each calculation in the model takes time. If the calculations in one step take longer than the sample period, the step overruns and the model no longer runs in real time. The function below adds a controllable amount of extra work to the model.

MATLAB Function Block: adding a controllable amount of computation to the model

```
function y = fcn(x, N)
% Repeats a floating-point calculation N times to load the processor
y = x;
for k = 1:N
    y = sqrt(y + 1);
end
```

1. In the Configuration Parameters, under Hardware Implementation, enable overrun detection and select pin 13 (the on-board LED) as the digital output that is set when an overrun occurs.
2. Add the function above to the model, with x taken from the press counter and N from a Constant block. Connect its output to a Terminator.
3. With T_s = 0.01 s, deploy the model with N = 10 and check that the on-board LED stays off.
4. Double N and redeploy, repeating until the on-board LED lights. Record the smallest N that causes an overrun in Table 3.
5. Repeat for T_s = 0.005 s and T_s = 0.02 s.

Table 3: *Computation Limit for Different Sample Times*

Sample time, T_s (s)	Largest N without overrun	Smallest N with overrun
0.005		
0.01		
0.02		

The largest N without an overrun should roughly double each time the sample time doubles, because twice as much time is available in each step. A controller must always finish its calculations within one sample period, so the sample time of a real controller is chosen with a margin below this limit.

Observe the PWM Output

1. Add a slow ramp to the PWM value so that the LED brightness rises from 0 to 255 over 10 s. Watch the LED closely for any flicker.
2. Wave the LED quickly from side to side in a dim room at a low brightness. Describe what you see, and explain it using the PWM frequency of the pin you chose.

4.0 Concluding Remarks

This practical exercise investigated how a Simulink model runs on the ATmega328P of the Arduino Uno. Using MATLAB, Simulink and ACE-Lab hardware, a push-button LED dimmer was built, and the sample time was shown to decide which input events are detected: presses shorter than the sample period can be missed.

Adding computation to the model showed that each step must finish within one sample period, or the model overruns and stops running in real time. A controller therefore needs a sample time that is short enough for the dynamics being controlled, but long enough for the processor to complete every calculation reliably.

Recommended Citation

If you have used this practical exercise in your teaching, learning or research, please cite it using the reference below, adapting the format as required by your preferred referencing style.

Pickering, J. E. (2026). Sample Time and Timing on the ATmega328P. ACE-Lab. Teaching Notes: ACE-Lab Practical Exercises, Version 1.0. Zenodo. https://zenodo.org/records/XXXXXXXX